

Architecture Decision Records

The following Architecture Decision Records document the major architectural decisions made for SALESTORM, along with the reasoning, trade-offs, and alternatives considered for each decision.

1. ADR-001 — SQL as the Authoritative Inventory Store

Context

The flash sale creates extreme contention around a very small inventory pool. Thousands of concurrent requests may compete for the same limited stock, making inventory correctness more important than maximizing raw write throughput.

Decision

Use MySQL/PostgreSQL as the authoritative source of inventory state. Redis will be used for caching, rate limiting, and non-authoritative read acceleration, but not as the source of truth for inventory allocation.

Rationale

Inventory allocation requires strong consistency and transactional guarantees. A relational database provides a reliable mechanism for performing atomic inventory updates while maintaining the core invariant that available stock must never become negative.

Maintaining a single authoritative inventory store also prevents inconsistencies between cached and actual inventory state.

Consequences

The inventory record may experience high contention during the flash sale. This is mitigated through short transactions, atomic updates, immediate rejection of unsuccessful reservations, cached read operations, and horizontal scaling of the stateless application tier.

Rejected Alternative

Using Redis as the inventory source of truth was rejected because cache failure, persistence behaviour, failover scenarios, and concurrent updates could introduce additional risks to inventory correctness.

2. ADR-002 — Atomic Conditional UPDATE over Optimistic Versioning

Context

Thousands of requests may compete for the final available units simultaneously. Separating the stock check from the stock update could allow multiple requests to observe the same quantity before any update is committed.

Decision

Use an atomic conditional update:

```
UPDATE inventory
SET available_quantity = available_quantity - 1
WHERE product_id = ?
AND available_quantity >= 1;
```

A successful update indicates that the inventory was allocated. Zero affected rows indicates that the requested stock is unavailable.

Rationale

The stock check and decrement are performed atomically within a single database operation. This eliminates application-level check-then-update races and avoids the excessive retry activity that optimistic